

R Bootcamp 2 - Data Manipulation with Tidyverse

Giovanni Castro Irizarry
UCLA M183/M136C

Spring 2025 Week 3

Review

Variables

Practice Q1.

Assign the variable `z` the difference between `y` and `x` with `y = 50` and `x = 20`.

Data Frames

Practice Q2.

What is the mean of the variable `mpg` in the `mtcars` dataset nested in the `car` package?

Practice Q3.

How many columns are in the `mtcars` dataframe?

Basic Data Manipulation in R

For practice this week, we are going to use the data from a mini-experiment. I have copied the code that imports the data in the code chunk below.

Please follow along if you'd like or work through this at your own pace.

I will note now that for our purposes in this class, most of our focus will be on unpacking and analyzing the statistical meaning of our analyses, not so much on a deep understanding of the computer science of what R does under the hood. We cannot neglect a basic understanding of how to code, since knowing how to write good code is essential to effectively using R as a tool for analysis.

Lastly, oftentimes you will run into strange errors or anomalous outputs from your code. Don't panic! The best thing for you to do in these situations is to google the problem you're having. You will very often find that others have posted about your issue on coding sites like stackoverflow.com and that the problem you are working on has been solved by someone else. Learning how to understand how code works is very important for taking other peoples' solutions and porting them to the problem you're working on.

Feel free to grab code from any source you find applicable! But if you do, please indicate the source of your code in either a comment near where you apply the code or in text near the relevant code chunk. Failing to do this is plagiarism just as with taking ideas from academic texts.

Importing and displaying data

Data we want to look at is in a csv file. Tidyverse package has `read.csv` command that allows you to download and read .csv files on R

First, let's clean the Environment with the following code:

```
rm(list=ls())
```

But honestly, it is easier to use the broom in Environment (top right quadrant)

Let's load tidyverse package to use some of the commands we will need for today.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Now let's import some data. But to do so, you need to understand what is a working directory. Think of a directory as a folder on your computer. When you're using R, it always has a "current working directory" — basically the folder it's currently looking in by default when you load or save files.

```
#Check working directory
getwd()
```

```
## [1] "/Users/giovanni/Downloads"
```

```
#How to set the working directory:
#setwd("/Users/giovanni/Downloads")
```

When working in RMarkdown, the working directory is always the folder where the .Rmd file is located

```
#Import the data from dropbox and save it in an object called experiment
experiment <- read_csv("moussa.csv")
```

```
## Rows: 232 Columns: 57
## -- Column specification -----
## Delimiter: ","
## chr (3): team, league, player_type
## dbl (54): treated, edu, church, income, status1, marital, isis_abuse, block,...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#read.csv reads csv files, readRDS() reads RDS files, read_sav()
```

```
# Show the mini experiment data
print(experiment)
```

```
## # A tibble: 232 x 57
##   team    league treated  edu church income status1 marital isis_abuse
##   <chr>   <chr>     <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>     <dbl>
## 1 team_1 league1         1     6     6     0       2       1         0
```

```
## 2 team_2 league1 0 2 5 0 1 0 0
## 3 team_5 league1 1 6 2 4 0 1 0
## 4 team_6 league2 1 6 2 1 1 0 1
## 5 team_9 league1 0 2 2 0 0 0 0
## 6 team_10 league2 1 2 2 2 0 0 0
## 7 team_12 league1 0 2 5 1 1 0 0
## 8 team_13 league2 0 6 5 0 0 0 0
## 9 team_17 league1 0 3 5 0 1 0 0
## 10 team_18 league2 0 3 2 0 0 1 0
## # i 222 more rows
## # i 48 more variables: player_type <chr>, block <dbl>, birth.year <dbl>,
## #   voted_muslim_extra <dbl>, own_group_preference <dbl>, donate_t1_new1 <dbl>,
## #   donate_t0_new1 <dbl>, dv_secular_t0 <dbl>, dv_secular_t1 <dbl>,
## #   dv_neigh_t0 <dbl>, dv_neigh_t1 <dbl>, dv_blame_t0 <dbl>, dv_blame_t1 <dbl>,
## #   areas_t1_new <dbl>, friends_t1_new <dbl>, proud_iraqi_t1_new <dbl>,
## #   secular_t1_new <dbl>, secular1_t1_new <dbl>, land_t1_new <dbl>, ...
```

Pipes and select()

Before analyzing the data, we want to look at the treatment group and their outcome. The `select()` function in the `dplyr` package allows you to select and view the variables of interest from the dataset. It returns a subset of the columns of the dataset that are specified.

```
# Load dplyr package to use some of the commands we will need for today
library(dplyr)
```

I first use the function to create a vector without using the pipe. Also, notice how I wrapped the column names inside the `c()` function.

```
# Select the treatment and a few of the variable columns
dplyr::select(experiment, c("treated", "donate_t1_new1", "edu", "income", "voted_muslim_extra"))
```

```
## # A tibble: 232 x 5
##   treated donate_t1_new1   edu income voted_muslim_extra
##   <dbl>         <dbl> <dbl> <dbl>         <dbl>
## 1     1             1     6     0             1
## 2     0             1     2     0             NA
## 3     1             1     6     4             1
## 4     1             1     6     1             1
## 5     0             1     2     0             0
## 6     1             0     2     2             1
## 7     0             1     2     1             0
## 8     0             0     6     0             0
## 9     0             0     3     0             0
## 10    0             0     3     0             0
## # i 222 more rows
```

We can do the same thing using the pipe operator `%>%`. It takes the object on the left-hand side and passes it as the first parameter (or argument) of the function on the right-hand side.

```
# Select the same columns using a pipe %>%
experiment %>% #from the dataset called `experiment`
  dplyr::select(c("treated", "donate_t1_new1", "edu", "income", "voted_muslim_extra")) #view these vari
```

```
## # A tibble: 232 x 5
##   treated donate_t1_new1   edu income voted_muslim_extra
##   <dbl>         <dbl> <dbl> <dbl>         <dbl>
```

```
## 1      1      1      6      0      1
## 2      0      1      2      0      NA
## 3      1      1      6      4      1
## 4      1      1      6      1      1
## 5      0      1      2      0      0
## 6      1      0      2      2      1
## 7      0      1      2      1      0
## 8      0      0      6      0      0
## 9      0      0      3      0      0
## 10     0      0      3      0      0
## # i 222 more rows
```

Explanation of pipes

Imagine you have this code:

```
summary(log(sqrt(c(1, 4, 9, 16))))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000  0.5199  0.8959  0.7945  1.1705  1.3863
```

This can be hard to read because you need to start from inside to outside. With a pipe you can write the same command in a different way, with the same results:

```
c(1, 4, 9, 16) |> sqrt() |> log() |> summary()
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000  0.5199  0.8959  0.7945  1.1705  1.3863
```

filter()

Sometimes before analyzing data we want to remove observations that are problematic for our analysis. A systematic way to do this is with the `filter()` command. This function takes as its parameters 1) the dataset and 2) a logical operator dictating which rows of the dataset to RETAIN.

One thing you might encounter is that missing observations can prevent you from finding statistical estimates on your dataset.

The simplest way to handle missing data is to simply delete observations with missing values of variables of interest - listwise deletion. Since the index is the dependent variable of interest with a missing value, let's remove the row with a missing value using `filter()`. I added some code that saves new resulting dataset as `experiment_clean` so that we can retain the cleaned up version for subsequent analyses.

There are two ways to check if there are NA values in a dataset: 1) If you want to evaluate if a value is NA, you should use the `is.na()` function instead of standard logical operators. `is.na()` evaluates to TRUE anywhere the object inside the parentheses takes a value of NA. 2) The logical NOT operator `!` turns all TRUE values into FALSE and all FALSE values into TRUE.

```
# Create a sample dataset with NA values
nums <- c(1, 2, NA, 5, 10, NA)
```

```
# A quick demonstration of is.na()
is.na(nums) #shows NA values as TRUE
```

```
## [1] FALSE FALSE  TRUE FALSE FALSE  TRUE
```

```
# A quick demonstration of !
!is.na(nums) #shows NA values as FALSE
```

```
## [1]  TRUE  TRUE FALSE  TRUE  TRUE FALSE
```

Check for NA's in the voted_muslim_extra variable

```
is.na(experiment$voted_muslim_extra)
```

```
## [1] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [13] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [37] TRUE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE FALSE TRUE FALSE
## [49] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE FALSE FALSE
## [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE FALSE
## [73] TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [85] FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE FALSE TRUE TRUE TRUE
## [97] TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE TRUE FALSE TRUE TRUE
## [121] TRUE TRUE FALSE TRUE TRUE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE TRUE FALSE TRUE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE TRUE TRUE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE
## [157] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [169] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE FALSE
## [181] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [193] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE
## [205] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [217] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [229] FALSE FALSE TRUE FALSE
```

Use `summary()` command to know how many TRUE and FALSE are in the voted_muslim_extra variable

```
summary(is.na(experiment$voted_muslim_extra))
```

```
## Mode FALSE TRUE
## logical 183 49
```

Use the `filter()` command with a logical operation to remove rows with an NA value

```
experiment_clean <- filter(experiment, !is.na(voted_muslim_extra))
```

Or, you can do the same using pipes

```
experiment_clean <- experiment %>% #create a new dataset `_clean`
  filter(!is.na(voted_muslim_extra)) #and get rid of any NAs
```

Check for NA's in the voted_muslim_extra variable in the new cleaned dataset

```
is.na(experiment_clean$voted_muslim_extra)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [157] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
## [169] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [181] FALSE FALSE FALSE
```

mutate()

In order to better process our data, we often need to transform it into a more appropriate form.

That's where `mutate()` comes in. It adds new columns (variables) to the dataset you're using. Most importantly, it can do so based on the values of other columns.

Lets make a new variable called "outcome"

```
# Check the variables of interest
experiment_clean$dv_secular_t0
```

```
## [1] 1.1096270 -1.6152937 1.1096270 1.1096270 -0.1779633 1.1096270
## [7] 1.1096270 -0.1779633 -0.1779633 -0.1779633 -1.6152937 -0.1779633
## [13] -1.6152937 1.1096270 -0.1779633 -0.1779633 1.1096270 -0.1779633
## [19] 1.1096270 1.1096270 1.1096270 -1.6152937 -0.1779633 -0.1779633
## [25] -0.1779633 -0.1779633 -0.1779633 -1.6152937 1.1096270 -1.6152937
## [31] -0.1779633 -1.6152937 -1.6152937 -1.6152937 -0.3277033 1.1096270
## [37] 1.1096270 -0.1779633 -0.3277033 1.1096270 1.1096270 1.1096270
## [43] -1.6152937 -1.6152937 1.1096270 -0.1779633 1.1096270 -0.1779633
## [49] -0.1779633 1.1096270 -1.6152937 -1.6152937 -0.3277033 -0.1779633
## [55] -0.1779633 -0.1779633 -1.6152937 1.1096270 1.1096270 1.1096270
## [61] -0.1779633 1.1096270 -0.1779633 1.1096270 1.1096270 -0.1779633
## [67] 1.1096270 -1.6152937 1.1096270 -1.6152937 1.1096270 1.1096270
## [73] -0.1779633 -0.1779633 1.1096270 1.1096270 -1.6152937 1.1096270
## [79] -0.1779633 -0.1779633 -0.1779633 -0.1779633 -0.1779633 -0.1779633
## [85] -0.1779633 1.1096270 -1.6152937 -1.6152937 -0.3277033 -1.6152937
## [91] 1.1096270 -0.3277033 -1.6152937 -0.1779633 -0.1779633 1.1096270
## [97] -1.6152937 -1.6152937 -0.1779633 -0.1779633 -0.3277033 -0.1779633
## [103] -1.6152937 1.1096270 -1.6152937 1.1096270 1.1096270 -0.1779633
## [109] -1.6152937 1.1096270 -0.3277033 1.1096270 -0.3277033 -0.1779633
## [115] -1.6152937 -0.1779633 -0.1779633 -1.6152937 1.1096270 1.1096270
## [121] 1.1096270 1.1096270 -1.6152937 -0.1779633 -1.6152937 1.1096270
## [127] -0.1779633 -1.6152937 -0.1779633 1.1096270 -0.1779633 1.1096270
## [133] -0.1779633 -1.6152937 -0.1779633 1.1096270 1.1096270 -0.1779633
## [139] 1.1096270 -0.1779633 1.1096270 -1.6152937 -0.1779633 -0.3277033
## [145] 1.1096270 -0.1779633 1.1096270 -0.1779633 1.1096270 1.1096270
## [151] 1.1096270 -1.6152937 1.1096270 -1.6152937 -0.1779633 -0.1779633
## [157] -0.1779633 1.1096270 -0.3277033 -1.6152937 -0.1779633 -0.1779633
## [163] 1.1096270 -0.3277033 1.1096270 -1.6152937 1.1096270 1.1096270
## [169] 1.1096270 1.1096270 1.1096270 -0.3277033 -1.6152937 1.1096270
## [175] 1.1096270 -0.1779633 1.1096270 -0.3277033 -0.3277033 1.1096270
## [181] 1.1096270 1.1096270 -0.3277033
```

```
experiment_clean$dv_blame_t0
```

```
## [1] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [7] -0.3522947 1.4436296 -0.3522947 1.4436296 -0.3522947 -0.3522947
## [13] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [19] -0.3522947 -0.3522947 -0.3522947 1.4436296 -0.3522947 -0.3522947
## [25] -0.3522947 -0.3522947 -0.3522947 1.4436296 3.7660042 3.7660042
## [31] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [37] -0.3522947 -0.3522947 -0.3522947 3.7660042 -0.3522947 -0.3522947
## [43] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
```

```

## [49] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [55] -0.3522947 3.7660042 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [61] -0.3522947 3.7660042 -0.3522947 -0.3522947 1.4436296 -0.3522947
## [67] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [73] -0.3522947 1.4436296 -0.3522947 3.7660042 -0.3522947 -0.3522947
## [79] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [85] -0.3522947 -0.3522947 1.9700799 -0.3522947 -0.3522947 -0.3522947
## [91] 1.4436296 -0.3522947 3.7660042 -0.3522947 3.7660042 -0.3522947
## [97] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [103] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [109] -0.3522947 1.4436296 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [115] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 1.4436296
## [121] 1.4436296 -0.3522947 -0.3522947 -0.3522947 -0.3522947 1.4436296
## [127] -0.3522947 3.7660042 -0.3522947 3.7660042 -0.3522947 -0.3522947
## [133] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [139] -0.3522947 1.9700799 -0.3522947 -0.3522947 -0.3522947 1.4436296
## [145] -0.3522947 -0.3522947 1.9700799 -0.3522947 -0.3522947 3.7660042
## [151] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [157] -0.3522947 -0.3522947 3.7660042 -0.3522947 -0.3522947 -0.3522947
## [163] 1.4436296 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [169] 1.4436296 -0.3522947 -0.3522947 1.4436296 -0.3522947 -0.3522947
## [175] -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947 -0.3522947
## [181] -0.3522947 -0.3522947 -0.3522947

```

```
experiment_clean$dv_neigh_t0
```

```

## [1] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 2.0968678
## [7] 1.2435171 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [13] -0.3297501 0.7139327 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [19] -0.3297501 0.7139327 -0.3297501 1.0531850 -0.3297501 -0.3297501
## [25] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [31] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [37] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [43] -0.3297501 0.7139327 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [49] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [55] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [61] -0.3297501 -0.3297501 -0.3297501 -0.3297501 0.7139327 -0.3297501
## [67] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [73] -0.3297501 1.0531850 -0.3297501 -0.3297501 -0.3297501 6.2332056
## [79] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [85] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [91] -0.3297501 -0.3297501 3.6701351 -0.3297501 -0.3297501 2.0968678
## [97] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [103] -0.3297501 1.2435171 -0.3297501 2.0968678 -0.3297501 -0.3297501
## [109] -0.3297501 -0.3297501 -0.3297501 -0.3297501 0.7139327 -0.3297501
## [115] -0.3297501 -0.3297501 1.0531850 -0.3297501 -0.3297501 -0.3297501
## [121] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [127] -0.3297501 -0.3297501 -0.3297501 -0.3297501 2.2872000 -0.3297501
## [133] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [139] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [145] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [151] 0.7139327 2.0968678 -0.3297501 1.2435171 -0.3297501 -0.3297501
## [157] -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [163] 0.7139327 -0.3297501 -0.3297501 -0.3297501 -0.3297501 -0.3297501
## [169] -0.3297501 0.7139327 -0.3297501 -0.3297501 -0.3297501 -0.3297501

```

```
## [175]  1.2435171 -0.3297501  2.2333204 -0.3297501  2.0968678  1.2435171
## [181] -0.3297501 -0.3297501 -0.3297501

dplyr::select(experiment_clean, c("dv_secular_t0", "dv_blame_t0", "dv_neigh_t0"))
```

```
## # A tibble: 183 x 3
##   dv_secular_t0 dv_blame_t0 dv_neigh_t0
##   <dbl>         <dbl>         <dbl>
## 1         1.11        -0.352        -0.330
## 2        -1.62        -0.352        -0.330
## 3         1.11        -0.352        -0.330
## 4         1.11        -0.352        -0.330
## 5        -0.178       -0.352        -0.330
## 6         1.11        -0.352         2.10
## 7         1.11        -0.352         1.24
## 8        -0.178         1.44        -0.330
## 9        -0.178       -0.352        -0.330
## 10        -0.178         1.44        -0.330
## # i 173 more rows
```

```
#Or with pipes
experiment_clean %>% dplyr::select(c("dv_secular_t0", "dv_blame_t0", "dv_neigh_t0"))
```

```
## # A tibble: 183 x 3
##   dv_secular_t0 dv_blame_t0 dv_neigh_t0
##   <dbl>         <dbl>         <dbl>
## 1         1.11        -0.352        -0.330
## 2        -1.62        -0.352        -0.330
## 3         1.11        -0.352        -0.330
## 4         1.11        -0.352        -0.330
## 5        -0.178       -0.352        -0.330
## 6         1.11        -0.352         2.10
## 7         1.11        -0.352         1.24
## 8        -0.178         1.44        -0.330
## 9        -0.178       -0.352        -0.330
## 10        -0.178         1.44        -0.330
## # i 173 more rows
```

Eyeballing it, it seems like the mean of the three numerical variables - dv_secular_t0, dv_blame_t0, and dv_neigh_t0.

Let's verify that with the mutate() command.

```
# Starting with the cleaned dataset from last time
experiment_mutated <- experiment_clean %>% #create a new dataset `_mutated` from `_clean`
  mutate(outcome = (dv_secular_t0 + dv_blame_t0 + dv_neigh_t0)/3) #and create a new variable
```

Or you can just add this new variable outcome to the experiment_clean dataset with \$

```
experiment_clean$outcome <- (experiment_clean$dv_secular_t0 + experiment_clean$dv_blame_t0 + experiment
```

summarise() and group_by()

The summarise() function is a tool to apply certain functions to columns of data to reduce them to single values. And I used it just now to count up the number of TRUE values in the validation check above. In other words, you can produce simple summary statistics (mean of a column, the standard deviation) in this simple-to-use function.

`group_by()` is frequently used in conjunction with `summarise()`, since it restricts summary operations to within values of the grouping variable. This is useful if we want to think about differences between the treatment and control groups. Remember to put this function BEFORE the `summarise()` function - it won't work otherwise!

Let's try to calculate the means and standard deviations of the growth mindset index scores for each group!

```
experiment_mutated %>% #from the dataset called `experiment_mutated`  
  group_by(treated) %>% #want to look at the observations based on treatment and control groups  
  summarise(mean = mean(outcome), #mean of the outcome  
            sd = sd(outcome)) #standard deviation of the outcome
```

```
## # A tibble: 2 x 3  
##   treated    mean    sd  
##   <dbl>    <dbl> <dbl>  
## 1      0  0.0372  0.617  
## 2      1 -0.00378 0.585
```